

COMP 3311 DATABASE MANAGEMENT SYSTEMS

LECTURE 14 MONGODB DBMS

QUERYING ARRAYS

- Array **elements** can be **accessed by their position** in the array using dot notation.
 - The array name is concatenated with a dot (.), a zero-based index position and a field name all of which are enclosed in quotes.

Query: Find the name and loan information for the clients whose first loan amount is less than \$50,000.

```
db.clients.find({'loans.0.amount': {$lt: 50000}},  
  {_id: 0, name: 1, loans: 1});
```

Query result: {name: 'Mary Kwan', loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
 {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}

The query searches for documents with a loans array field whose **first element** (i.e., the element at index position 0) is an object with an amount field that is less than \$50,000.

Note that a query result document may have additional loans (e.g., loan L-434) with an amount field that is greater than or equal to \$50,000.

MQL Note

Array elements cannot be accessed using the dot notation in a **find** method *projection* (see operators **\$arrayElemAt** and **\$first**).

QUERYING ARRAYS (CONTD)

- The **fields of documents** in an array can be **accessed by name** using the dot notation.
 - The array name is concatenated with a dot (.) and the field name all of which are enclosed in quotes.

Query: Find the name and loans of clients where the client has a loan with amount greater than \$75,000.

```
db.clients.find({'loans.amount': {'$gt': 75000}},  
  {'_id': 0, name: 1, loans: 1});
```

Query result: {name: 'Steven Chan', loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},
 {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}

The query searches for documents with a loans array field that contains **at least one element** (i.e., at least one object) where the amount field is greater than \$75,000.

Note that a query result document may also have other loan array objects with an amount field that is less than or equal to \$75,000 (e.g., loan L-435).

✎ **Equality matches to select a specific array element require an exact match of the element on all fields including field order.**

QUERYING ARRAYS (CONTD)

Query: Find the name and loans of clients where the client has a loan at the Festival Walk branch with amount less than \$50,000.

```
db.clients.find({'loans.branch': 'Festival Walk', 'loans.amount': {'$lt': 50000}},  
  {'_id': 0, name: 1, loans: 1});
```

Query result:

```
{name: 'Steven Chan', loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},  
  {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}
```

```
{name: 'Mary Kwan', loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},  
  {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}}
```

← matches amount field

← matches branch field

```
{name: 'Pam Du', loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}
```

This query will not obtain the desired result since it matches documents where the loans array field contains **at least one element** (i.e., at least one object) where the branch field value is Festival Walk and **at least one element** (i.e., at least one object) where the amount field value is less than \$50,000.

However, **the branch and amount fields do not have to be in the same array element!**
The same result is obtained if the **\$and** operator is used since the **\$and** operator is not intended for querying on multiple fields of an array of objects.

 **An and condition on array elements can be satisfied by matches in different array elements.**

QUERYING ARRAYS:

THE \$NE COMPARISON OPERATOR

Query: Find the name and loans of clients where the client has a loan with amount not equal to \$55,000.

```
db.clients.find({'loans.amount': {'$ne': 55000}},  
  {'_id': 0, name: 1, loans: 1});
```

Query result: {name: 'Mary Kwan', loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
 {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}
 {name: 'Isaac Li'}

This query searches for documents where the loans array field contains **at least one element** (i.e., at least one object) whose amount field is equal to \$55,000 and *removes* that document from the query result even though the loans array field may contain other objects whose amount field is not equal to \$55,000 (e.g., Steven Chan). While this may seem counterintuitive, it makes sense if we consider that the query

```
db.clients.find({'loans.amount': {'$eq': 55000}},  
  {'_id': 0, name: 1, loans: 1});
```

returns documents where the loans array field contains **at least one element** (i.e., at least one object) whose amount field is equal to \$55,000. Using similar semantics, the **\$ne** operator *removes* documents where the loans array field contains **at least one element** (i.e., at least one object) whose amount field is equal to \$55,000.

ARRAY QUERY OPERATOR: \$SELEMMATCH

arrayField: {**\$selemMatch**: {*predicate*₁, *predicate*₂, ..., *predicate*_n}

The **\$selemMatch** operator matches documents that contain *arrayField* and this array has **at least one element that satisfies all the specified predicates**.

- If *arrayField* is not an array but it matches the query conditions, then it is not included in the query result.

👉 **\$selemMatch can only be used to match array elements.**

Query: Find the name and loans of clients where the client has a loan at the Festival Walk branch with amount greater than \$50,000.

```
db.clients.find({loans: {$selemMatch: {amount: {$gt: 50000},  
                                     branch: 'Festival Walk'}}},  
  {_id: 0, name: 1, loans: 1});
```

Query result:

```
{name: 'Steven Chan', loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},  
                             {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]},  
{name: 'Mary Kwan', loans: [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},  
                             {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]},  
{name: 'Pam Du', loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}
```

ARRAY QUERY OPERATOR: \$SELEMMATCH (CONT'D)

The **\$selemMatch** operator can be used in a **find** method *projection* to return only those query result array elements that match the specified predicate.

✎ Only the first element in the query result array that matches the predicate is returned.

Query: Find the name of clients who have a loan amount greater than \$50,000 and include one of these loans in the result.

```
db.clients.find({'loans.amount': {$gt: 50000}},  
  {'_id: 0, name: 1, loans: {$selemMatch: {amount: {$gt: 50000}}}});
```

Query result: {name: 'Steven Chan', loans: [{loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'}]}
{name: 'Mary Kwan', loans: [{loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}
{name: 'Pam Du', loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}

This query finds all clients with loans whose amount is greater than \$50,000 and returns only the first loan (i.e., the first element) in the loans array field whose amount is greater than \$50,000 for each client. Other loans of a client whose amount also matches the **\$selemMatch** predicate are not in the query result (e.g., loan L-435 for Steven Chan).

Note that if the **find** filter condition is omitted, then all clients are returned and those with no loan greater than \$50,000 will display only the name (e.g., Isaac Li).

ARRAY QUERY OPERATOR: **\$all**

arrayField: {**\$all**: [*value*₁, *value*₂, ..., *value*_{*n*}]}

The **\$all** operator selects the documents where *arrayField* is an **array of values** and the array contains **all** the elements specified in the array of *values* (in any order).

Query: Find the name and tags of the clients with tags 'vip' and 'senior'.

```
db.clients.find({tags: {$all: ['vip', 'senior']}},  
  {_id: 0, name: 1, tags: 1});
```

Query result: {name: 'Steven Chan', tags: ['senior', 'vip']}

👉 **\$all is equivalent to \$and of the specified values.**

```
db.clients.find({$and: [{tags: 'vip'}, {tags: 'senior'}]},  
  {_id: 0, name: 1, tags: 1});
```


ARRAY QUERY OPERATOR: \$SIZE

arrayField: {**\$size**: *number*}

The **\$size** operator matches *arrayField* if it has exactly *number* elements.

Query: Find the name of the clients who have exactly two accounts.

```
db.clients.find({accounts: {$size: 2}},  
  {_id: 0, name: 1});
```

Query result: {name: 'Mary Kwan'}

The query returns all documents where the accounts array field has exactly two elements (i.e., the client has exactly two accounts).

Query: Find the name of the clients who have only one loan.

```
db.clients.find({loans: {$size: 1}},  
  {_id: 0, name: 1});
```

Query result: {name: 'Pam Du'}

ARRAY EXPRESSION OPERATOR: \$SIZE

`{ $size: array }`

The `$size` operator, when used in a `find` method *projection*, returns the number of elements in *array*.

😞 An **error** is raised if *array* is absent in the document. 😞

Query: Find, for each client who holds one or more accounts, their name and the number of accounts they hold.

```
db.clients.find({accounts: { $exists: true },  
  {_id: 0, name: 1, numAccounts: { $size: '$account' }});
```

Query {name: 'Steven Chan', numAccounts: 3}
result: {name: 'Mary Kwan', numAccounts: 2}
 {name: 'Isaac Li', numAccounts: 1}

SQL Note

Since the `$size` array expression operator raises an error if the operand is not an array (or does not exist), the *queryFilter* is needed to remove documents with no accounts array field. Alternatively, a check can be made to determine whether the field is an array (or exists) and appropriate action taken based on the result of the check (see next slide).

ARRAY EXPRESSION OPERATOR: \$ISARRAY

\$isArray: *expression*

The **\$isArray** operator determines if *expression* is an array and returns **true** if *expression* is an array and **false** if *expression* is not an array or does not exist in the document.

Query: Find the name, number of accounts and number of loans for each client.

```
db.clients.find({ },
  { _id: 0, name: 1, numAccounts: { $cond: { if: { $isArray: '$accounts' },
    then: { $size: '$accounts' },
    else: 0 } },
    numLoans: { $cond: { if: { $isArray: '$loans' },
    then: { $size: '$loans' },
    else: 0 } } } } );
```

Query result: {name: 'Steven Chan', numAccounts: 3, numLoans: 2}
{name: 'Mary Kwan', numAccounts: 2, numLoans: 2}
{name: 'Isaac Li', numAccounts: 1, numLoans: 0}
{name: 'Pam Du', numAccounts: 0, numLoans: 1}

A zero count is returned for clients with no account or no loan (i.e., the respective field is absent).

ARRAY EXPRESSION OPERATOR: **\$ARRAYELEMENTAT**

{*\$arrayElemAt*: [*array*, *index*]}

The ***\$arrayElemAt*** operator returns the *array* element at the specified *index*.

- The first array element is at index zero.
- If *index* is zero or a positive integer, the element at the index position counting from the start of the array is returned.
- If *index* is a negative integer, the element at the index position counting backward from the end of the array is returned.
- if *index* exceeds the array bounds, no result is returned.
- If *array* is absent, *null* is returned.

Query: Find, for each client, their name and their second loan.

```
db.clients.find({ },  
  {_id: 0, name: 1, secondLoan: {$arrayElemAt: ['$loans', 1]}});
```

Query result: {name: 'Steven Chan', secondLoan: {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}}
{name: 'Mary Kwan', secondLoan: {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}}
{name: 'Isaac Li', secondLoan: null}
{name: 'Pam Du'}

ARRAY EXPRESSION OPERATORS: \$IN, \$NIN

field: {\$in: [value₁, value₂, ..., value_n]}

field: {\$nin: [value₁, value₂, ..., value_n]}

The \$in (\$nin) operator returns documents where the *field* value equals any value in the specified array (is not in the specified array or does not exist).

- If *field* is an array, then \$in (\$nin) returns documents where *field* contains at least one element (contains no element) that matches a value in the specified array.

Query: Find the name and tags of clients who are senior or vip.

```
db.clients.find({tags: {$in: ['senior', 'vip']}},  
  {_id: 0, name: 1, tags: 1});
```

The tags array field must contain either the value senior or vip (or both).

Query: {name: 'Steven Chan', tags: ['senior', 'vip']}

result: {name: 'Mary Kwan', tags: ['senior']}
 {name: 'Pam Du', tags: ['vip']}

Query: Find the name and tags of clients who are not senior or vip.

```
db.clients.find({tags: {$nin: ['senior', 'vip']}},  
  {_id: 0, name: 1, tags: });
```

The query result contains clients whose tags array field does not contain the values senior or vip as well as clients whose tags array field is null or absent.

Query result: {name: 'Isaac Li'}



ARRAY EXPRESSION OPERATORS: \$FIRST, \$LAST

{*\$first*: *array*}

{*\$last*: *array*}

The ***\$first*** (***\$last***) operator returns the first (last) element of *array*.

➤ If *array* has no elements or is absent, then **null** is returned.

Query: Find the name and first account of each client.

```
db.clients.find({  
  {_id: 0, name: 1, firstAccount: {$first: '$accounts'}}});
```

For clients who hold no account, the firstAccount field value is **null**.

Query {name: 'Steven Chan', firstAccount: {accountNo: 'A-117', balance: 35000, branch: 'Star House'}}
result: {name: 'Mary Kwan', firstAccount: {accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'}}
 {name: 'Isaac Li', firstAccount: {accountNo: 'A-120', balance: 80000, branch: 'Star House'}}
 {name: 'Pam Du', firstAccount: **null**}

Query: Find the name and last loan of the clients who have loans.

```
db.clients.find({loans: {$exists: true}},  
  {_id: 0, name: 1, lastLoan: {$last: '$loans'}});
```

Clients who have no loan will not appear in the query result.

Query {name: 'Steven Chan', lastLoan: {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}}
result: {name: 'Mary Kwan', lastLoan: {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}}
 {name: 'Pam Du', lastLoan: {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}}
 {name: 'Pam Du', lastLoan: **null**}

ARRAY EXPRESSION OPERATORS: **\$FIRSTN**, **\$LASTN**

\$firstN: {input: *array*, n: *integer*} **\$lastN**: {input: *array*, n: *integer*}

The **\$firstN** (**\$lastN**) operator returns *integer* elements from the beginning (end) of *array*.

- If *array* has fewer than *integer* elements, then all elements are returned.
- *null* values are not removed from the result.

😞 An **error** is raised if *array* is absent in the document. 😞

Query: Find the name and first two accounts of each client.

```
db.clients.find({  
  {  
    _id: 0, name: 1,  
    firstTwoAccounts: {  
      $cond: {  
        if: {  
          $isArray: '$accounts',  
          then: {  
            $firstN: {  
              input: '$accounts',  
              n: 2,  
            },  
            else: 'no accounts',  
          },  
        },  
      },  
    },  
  },  
});
```

Query result:

```
{name: 'Steven Chan', firstTwoAccounts: [{accountNo: 'A-117', balance: 35000, branch: 'Star House'},  
  {accountNo: 'A-120', balance: 80000, branch: 'Star House'}]},  
{name: 'Mary Kwan', firstTwoAccounts: [{accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},  
  {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'}]},  
{name: 'Isaac Li', firstTwoAccounts: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'}]},  
{name: 'Pam Du', firstTwoAccounts: 'no accounts'}
```

ARRAY EXPRESSION OPERATORS: \$MINN, \$MAXN

\$minN: {input: *array*, n: *integer*} { **\$maxN**: {input: *array*, n: *integer*}}

The **\$minN** (**\$maxN**) operator returns *integer* number of the smallest (largest) values in *array*.

- If *array* has fewer than *integer* elements, then all elements are returned.
- *null* values are removed from the result.

😞 An **error** is raised if *array* is absent in the document. 😞

Query: Find, for each client, the name and the minimum account balance of all the client's accounts.

```
db.clients.find({ },
  { _id: 0, name: 1,
    minBalance: { $cond: { if: { $isArray: '$accounts' },
                          then: { $minN: { input: '$accounts.balance', n: 1 } },
                          else: 'no accounts' } } } });
```

Query result: {name: 'Steven Chan', minBalance: [35000]}
 {name: 'Mary Kwan', minBalance: [45000]}
 {name: 'Isaac Li', minBalance: [80000]}
 {name: 'Pam Du', minBalance: 'no accounts'}

ARRAY EXPRESSION OPERATORS: **\$FILTER**

The **\$filter** operator selects only those elements in *array* that match the specified *predicate* and returns them in an array.

Query: Find the name of the clients and their accounts where the account balance is greater than \$60,000.

```
db.clients.find({ },
  { _id: 0, name: 1,
    largeAccounts: { $filter: { input: '$accounts',
                               as: 'acct',
                               cond: { $gt: ['$acct.balance', 60000] } } } } );
```

Query result:

```
{name: 'Steven Chan', largeAccounts: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'},
                                     {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'}]}
{name: 'Mary Kwan', largeAccounts: [ ]}
{name: 'Isaac Li', largeAccounts: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'}]}
{name: 'Pam Du', largeAccounts: null}
```

ARRAY EXPRESSION OPERATORS: **\$FILTER** (CONTD)

\$filter: {input: *array*, as: *variableName*, cond: *predicate*, limit: *integer*}

The **\$filter** operator selects only those elements in *array* that match the specified *predicate* and returns them in an array.

- If *array* is *null* or absent, *null* is returned.
- The *as* field is optional; *variableName* represents each individual element of *array* and can be used to refer to the array element currently being processed using a **\$\$** prefix (i.e., **\$\$***variableName*). If the *as* field is absent, *variableName* defaults to *this*.
- The *limit* field is optional; it restricts the number of matching array elements to return to at most *integer*.
- The returned array elements are in the original order.

😞 An **error** is raised if *input* is not an array. 😞

ARRAY EXPRESSION OPERATORS: \$MAP

The **\$map** operator applies *expression* to each element in *array* and returns an array with the applied results.

Query: Show all account balances increased by 5%.

```
db.clients.find({accounts: {$exists: true}},
  {_id: 0, name: 1,
    increasedBalance: {$map: {input: '$accounts',
      in: {$multiply: ['$$this.balance', 1.05]}}}});
```

Query {name: 'Steven Chan', increasedBalance: [36750, 84000, 78750]}
result: {name: 'Mary Kwan', increasedBalance: [57750, 47250]}
 {name: 'Isaac Li', increasedBalance: [84000]}
 {name: 'Pam Du', increasedBalance: null}

The database is not updated; only the query result is changed.

ARRAY EXPRESSION OPERATORS: \$MAP

\$map: {input: *array*, as: *variableName*, in: *expression*}

The **\$map** operator applies *expression* to each element in *array* and returns an array with the applied results.

- If *array* is *null* or absent, *null* is returned.
- The *as* field is optional; *variableName* represents each individual element of *array* and can be used to refer to the array element currently being processed using a **\$\$** prefix (i.e., **\$\$***variableName*). If the *as* field is absent, *variableName* defaults to *this*.

😞 An **error** is raised if *input* is not an array. 😞

ARRAY EXPRESSION OPERATORS: \$REDUCE (CONTD)

The **\$reduce** operator applies the *in expression* to each element in *array* and combines the elements into a single value.

Query: Find the name and tags of the clients who have tags.
Return the result tags as a single string.

```
db.clients.find({
  _id: 0, name: 1,
  tags: { $reduce: {input: '$tags',
                    initialValue: '', // empty initial string
                    in: { $cond: {if: { $eq: [{ $indexOfArray: ['$tags', '$$this']}, 0]},
                                // add the first tag
                                then: { $concat: ['$value', '$$this']},
                                // add a separator and subsequent tags
                                else: { $concat: ['$value', ',', '$$this'] }
                              }
                    }
  }
});
```

Query {name: 'Steven Chan', tags: 'senior, vip'}
result: {name: 'Mary Kwan', tags: 'senior'}
 {name: 'Isaac Li', tags: null}
 {name: 'Pam Du', tags: 'vip'}

ARRAY EXPRESSION OPERATORS: \$REDUCE

\$reduce: {input: *array*, initialValue: *expression*, in: *expression*}

The **\$reduce** operator applies the *in expression* to each element in *array* and combines the elements into a single value.

- If *input* is *null* or absent, *null* is returned.
- If *array* is empty, *initialValue* is returned.
- During the evaluation of the *in expression* two variables are available which are accessed using a \$\$ prefix:
 - *value* represents the cumulative value of the *in expression*.
 - *this* refers to the element being processed.

😞 An **error** is raised if the *input* is not an array. 😞

👉 **\$reduce can be used to transform array elements into a single string.**

ARRAY EXPRESSION OPERATOR: \$CONCATARRAYS

The `$concatArrays` operator concatenates two or more arrays and returns the concatenated array.

➤ If *any* input array is `null` or absent, then `null` is returned as the result.

Query: Find for each client their name and the information of all their accounts and all their loans as a single array.

```
db.clients.find({  
  { _id: 0, name: 1, accountsAndLoans: { $concatArrays: ['$accounts', '$loans'] } } });
```

This query will not obtain the desired result. Since not every client has an account or a loan, the result will be `null` for those clients who have only an account or only a loan (i.e., their account or loan will not appear in the query result).

For the example database, the `accountsAndLoans` field for Isaac Li and Pam Du will be `null`.

👉 **Need to check if an input array is `null` and handle accordingly.**

ARRAY EXPRESSION OPERATOR: \$CONCATARRAYS (CONTD)

Query: Find for each client their name and the information of all their accounts and all their loans as a single array.

```
db.clients.find({  
  {_id: 0, name: 1,  
  accountsAndLoans: {$concatArrays: [{$ifNull: [ '$accounts', [ ] ]},  
                                         {$ifNull: [ '$loans', [ ] ] } ]}});
```

Query result:

```
{name: 'Steven Chan', accountsAndLoans: [{accountNo: 'A-117', balance: 35000, branch: 'Star House'},  
                                         {accountNo: 'A-120', balance: 80000, branch: 'Star House'},  
                                         {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'},  
                                         {loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'},  
                                         {loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}}]  
{name: 'Mary Kwan', accountsAndLoans: [{accountNo: 'A-215', balance: 55000, branch: 'Diamond Hill'},  
                                         {accountNo: 'A-322', balance: 45000, branch: 'Pacific Place'},  
                                         {loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},  
                                         {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}}]  
{name: 'Isaac Li', accountsAndLoans: [{accountNo: 'A-120', balance: 80000, branch: 'Star House'}]  
{name: 'Pam Du', accountsAndLoans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'}]}
```


ARRAY EXPRESSION OPERATOR: \$SLICE

arrayField: {**\$slice**: *number*}

The **\$slice** operator specifies the *number* of elements in *arrayField* to return in the query result.

- If *number* is positive, the first *number* elements are returned.
- If *number* is negative, the last *number* elements are returned.
- If *number* is greater than the number of array elements, all elements are returned.

Query: Find the first two loans of client Mary Kwan.

```
db.clients.find({name: 'Mary Kwan'},  
  {_id: 0, name: 1, loans: {$slice: 2}});
```

Query result: {name: 'Mary Kwan', loans : [{loanNo: 'L-321', amount: 40000, year: '2024', branch: 'Pacific Place'},
 {loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}

Query: Find the last loan of client Mary Kwan.

```
db.clients.find({name: 'Pat Lee'},  
  {_id: 0, name: 1, loans: {$slice: -1}});
```

Query result: {name: 'Mary Kwan', loans : [{loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}

ARRAY EXPRESSION OPERATOR: \$SLICE (CONTD)

arrayField: {**\$slice**: [*numberToSkip*, *numberToReturn*]}

If **\$slice** is given an array of two numbers, then *numberToSkip* specifies the number of elements to skip in the query result and *numberToReturn* specifies the number of elements to return in the query result.

- If *numberToSkip* is positive, then *numberToSkip* elements starting from the beginning of the array are skipped.
- If *numberToSkip* is negative, then *numberToSkip* elements going backward from the end of the array are skipped and the following *numberToReturn* are returned.
- If *numberToReturn* is greater than the number of array elements, then only the number of array elements are returned.
- If *numberToSkip* is greater than the number of array elements, then an empty array is returned.

Query: Find the last loan of client Mary Kwan.

```
db.clients.find({name: 'Mary Kwan',  
  {_id: 0, name: 1, loans: {$slice: [-1, 1]}});
```

Count backwards one element from the end of the array and return one element starting from that position.

Query result: {name: 'Mary Kwan', loans : [{loanNo: 'L-434', amount: 52000, year: '2025', branch: 'Festival Walk'}]}

ARRAY EXPRESSION OPERATOR: \$SORTARRAY

\$sortArray: {input: *array*, sortBy: *sortOrder*}

The **\$sortArray** operator orders *array* according to the specified fields and sort orders.

- *sortOrder* is specified as **1** (ascending) or **-1** (descending) and is evaluated from left to right.
- If the array elements are objects, then they can be sorted by either an object field or a subfield of the object (e.g., sortBy: {*field*₁: *sortOrder*₁, ...}).

Query: Find, for each client, the name and their accounts and loans sorted by balance and amount, respectively, ascending.

```
db.clients.find({  
  {_id: 0, clientId: 1,  
    accounts: {$sortArray: {input: '$accounts', sortBy: {balance: 1}}},  
    loans: {$sortArray: {input: '$loans', sortBy: {amount: 1}}}});
```

Query result: {name: 'Steven Chan', accounts: [{accountNo: 'A-117', balance: 35000, branch: 'Star House'},
 {accountNo: 'A-301', balance: 75000, branch: 'Pacific Place'},
 {accountNo: 'A-120', balance: 80000, branch: 'Star House'}],
 loans: [{loanNo: 'L-435', amount: 55000, year: '2025', branch: 'Festival Walk'},
 {loanNo: 'L-171', amount: 90000, year: '2024', branch: 'Star House'}]}

SET EXPRESSION OPERATORS

`$setDifference`: [*array*₁, *array*₂]

Takes two arrays, treating them as sets, and returns an **unordered** array containing the **unique** elements that only appear in *array*₁.

`$setIntersection`: [*array*₁, *array*₂, ...]

Takes two or more arrays, treating them as sets, and returns an **unordered** array that contains the **unique** elements that appear in every input array.

➤ If there are no common elements, then an empty array is returned.

`$setUnion`: [*array*₁, *array*₂, ...]

Takes two or more arrays, treating them as sets, and returns an **unordered** array containing the **unique** elements that appear in any input array.

👉 **Set expression operators are most commonly used in the aggregation framework.**

SET EXPRESSION OPERATORS (CONTD)

{`$allElementsTrue`: [`array`]}

Evaluates `array` as a set and returns `true` if no element in the array is `false`. Otherwise, returns `false`. An empty array returns `true`.

{`$anyElementTrue`: [`array`]}

Evaluates `array` as a set and returns `true` if any of the elements are `true` and `false` otherwise. An empty array returns `false`.

{`$setEquals`: [`array`₁, `array`₂, ...]}

Compares two or more arrays, treating them as sets, and returns `true` if they have the same distinct elements and `false` otherwise.

{`$setIsSubset`: [`array`₁, `array`₂]}

Takes two arrays, treating them as sets, and returns `true` when `array`₁ is a subset of `array`₂, including when the `array`₁ equals `array`₂, and `false` otherwise.

DUPLICATE FIELD VALUE REMOVAL: DISTINCT METHOD

`distinct(field, queryFilter, collation)`

The **distinct** method finds the unique values for *field* in a collection that satisfies an optional *queryFilter* predicate; it returns the results in an array. An optional *collation* specifies the language-specific rules for string comparison.

Query: Find the unique districts of branches.

```
db.branches.distinct('district');
```

Includes **null** as a value if *field* has a **null** value.

SQL Note

Quotes are required around all field names for the **distinct** method.

Query: Find the unique branches of accounts.

```
db.clients.distinct('accounts.branch');
```

Query: Find the unique districts of clients with a rating greater than 8.

```
db.clients.distinct('district', {rating: {$gt: 8}});
```

NUMBER OF QUERY RESULTS: COUNT METHOD

`count( )`

The `count` method returns the number of documents in a query result.

Query: Find the number of documents in the clients collection.

```
db.clients.find( ).count();
```

Query: Find the number of clients who have loans.

```
db.clients.find({loans: {exists: true}}).count();
```

Query: Find the number of clients who have an account, but do not have a loan.

```
db.clients.find({accounts: {exists: true}, loans: {exists: false}}).count();
```

ORDERING QUERY RESULTS: SORT METHOD

```
sort({field1: sortOrder, field2: sortOrder, ..., fieldn: sortOrder});
```

The **sort** method orders the documents from a collection according to the specified fields.

- The *sortOrder* is specified as **1** (ascending) or **-1** (descending) and is evaluated from left to right.

Query: Find the client information for clients who have a loan at the Festival Walk branch. Order the result first by credit rating descending and then by name ascending.

```
db.clients.find({'loans.branch': 'Festival Walk'},  
  {_id: 0, accounts: 0, loans: 0}).sort({rating: -1, name: 1});
```

SQL Notes

1. Sorting can only be performed on fields in the input documents, not on computed fields in a *projection*.
2. The **sort** method is always applied before the projection of the fields in the original document.
(1 and 2 do not apply to the **\$sort** aggregation stage discussed later.)
3. Documents in which the sort field is **null** or **absent** appear as the smallest values in the sort order.

ORDERING QUERY RESULTS:

SORT METHOD (CONT'D)

- When sorting on the field of an object inside an array, the results are ordered based on the **smallest/largest value of the field of any object** in the array.

Query: Find, for each client, their name and account amounts.
Order clients based on their account balance in ascending order.

```
db.clients.find({ },  
  { _id: 0, name: 1, 'accounts.balance': 1 }).sort({'accounts.balance': 1});
```

Query {name: 'Pam Dui'}

result: {name: 'Steven Chan', accounts: [{balance: 35000}, {balance: 80000}, {balance: 75000}]},
 {name: 'Mary Kwan', accounts: [{balance: 55000}, {balance: 45000}]},
 {name: 'Isaac Li', accounts: [{balance: 80000}]}

In the query result, clients with no accounts field appear first, followed by clients that have the smallest account balance of all accounts including their own, etc. If a client has several accounts, then the account with the smallest balance is used to determine the sort order of the query result.

 **The \$sortBy operator is used to sort array elements.**



LIMITING QUERY RESULTS:

LIMIT METHOD

limit(*number*)

The **limit** method returns *number* documents from the query result.

Query: Find the name and credit rating of any three clients with the highest credit ratings. Order the result first by credit rating descending and then by name ascending.

```
db.clients.find({ },  
  {_id: 0, name: 1, rating: 1}).sort({rating: -1, name: 1}).limit(3);
```

Query {name: 'Steven Chan', rating: 9}
result: {name: 'Pam Du', rating: 8}
 {name: 'Isaac Li', rating: 6}

LIMITING QUERY RESULTS:

LIMIT METHOD (CONTD)

Query: Find the name and credit rating of any three clients with the lowest credit ratings. Order the result first by credit rating ascending and then by name ascending.

```
db.clients.find({rating: {$exists: true}, rating: {$ne: null}},  
  {_id: 0, name: 1, rating: 1}).sort({rating: 1, name: 1}).limit(3);
```

Query {name: 'Isaac Li', rating: 6}
result: {name: 'Mary Kwan', rating: 6}
 {name: 'Pam Du', rating: 8}

MQL Note

The *queryFilter* predicate is needed to eliminate clients whose rating is absent or null since, in **MQL**, absent and null values are the smallest values in the sort order.

LIMITING QUERY RESULTS: SKIP METHOD

`skip(number)`

The `skip` method skips *number* documents in the query result. It is normally used in conjunction with the `limit` method.

Query: Find the name and credit rating of the third client with the highest credit rating.

```
db.clients.find({ }, { _id: 0, name: 1, rating: 1 }).sort({ rating: -1 }).skip(2).limit(1);
```

```
db.clients.find({ }, { _id: 0, name: 1, rating: 1 }).skip(2).sort({ rating: -1 }).limit(1);
```

```
db.clients.find({ }, { _id: 0, name: 1, rating: 1 }).limit(1).skip(2).sort({ rating: -1 });
```

Query result: {name: 'Mary Kwan', rating: 6}

MQL Notes

1. These queries all return the same result since the `sort` method is always performed first, followed by the `skip` method and finally the `limit` method *regardless of the order in which they are specified*. This is not true for the aggregation framework (see next lecture).
2. To include ties, i.e., all clients with the third highest credit rating, the aggregation framework needs to be used.

SCHEMA VALIDATION

- **MongoDB** uses **JSON Schema** to specify field validation rules, such as allowed data types and value ranges.
 - A **JSON Schema** is defined using the **\$jsonSchema** keyword.
- The data type of a field is specified using the **bsonType** (binary **JSON**) keyword.
 - Some common valid **BSON** data types are **array**, **binData**, **bool**, **date**, **decimal**, **double**, **integer**, **long**, **null**, **object**, **regex** and **string**.
- Schema validation can be specified either before or after a collection is created.
 - This allows for flexible initial collection creation with no validation.

JSON SCHEMA EXAMPLE

Validation for clients collection

```
validator:
{$jsonSchema: {
  bsonType: 'object',
  required: ['clientId', 'name', 'hkid', 'address'], // required fields
  properties: {
    clientId: {bsonType: 'int'},
    name: {bsonType: 'string'},
    hkid: {bsonType: 'string'},
    address: {bsonType: 'string'},
    district: {bsonType: 'string', enum: ['Central and Western', 'Islands']},
    rating: {bsonType: 'int', minimum: 1, maximum: 10,
      description: 'rating must be an integer between 1 and 10'} // message when error is raised
    accounts: {bsonType: 'array',
      required: ['accountNo', 'balance', 'branch'],
      properties: {
        accountNo: {bsonType: 'string'},
        balance: {bsonType: 'double', minimum: 0},
        branch: {bsonType: 'string'}}},
    loans: {bsonType: 'array',
      required: ['loanNo', 'amount', 'year', 'branch'],
      properties: {
        loanNo: {bsonType: 'string'},
        amount: {bsonType: 'double', minimum: 0},
        year: {bsonType: 'string'},
        branch: {bsonType: 'string'}}},
    tags: {bsonType: 'array'}}
}}
```



MONGODB JSON SCHEMA: KEYWORDS

Keyword	Meaning
<code>bsonType</code>	the JSON type of a field (possible specified as an array of JSON types)
<code>description</code>	describes the schema; included in output if an error occurs
<code>enum</code>	an array that enumerates all possible values of a field
<code>exclusiveMaximum</code>	a boolean that, if true, specifies that a number field maximum is exclusive
<code>exclusiveMinimum</code>	a boolean that, if true, specifies that a number field minimum is exclusive
<code>maximum</code>	specifies the maximum value of a numeric field
<code>maxItems</code>	specifies the maximum length of an array
<code>maxLength</code>	specifies the maximum length of a field
<code>minimum</code>	specifies the minimum value of a numeric field
<code>minItems</code>	specifies the minimum length of an array
<code>minLength</code>	specifies the minimum length of a field
<code>multipleOf</code>	specifies that a numeric field's value must be a multiple of the specified value
<code>pattern</code>	specifies that a field must match the regex regular expression
<code>properties</code>	specifies, as an object, the fields of an object and their data types
<code>required</code>	specifies, as an array of strings, the required fields of an object
<code>uniqueItems</code>	a boolean that, if true, specifies that each item in an array must be unique